



Jetpack Architecture Components

Ha Noi August - 2023

OPEN MIND - OPEN DOORS



Contents

1. What is Architecture Components?

2. Introduction to components in Android Jetpack

3. Navigation Component

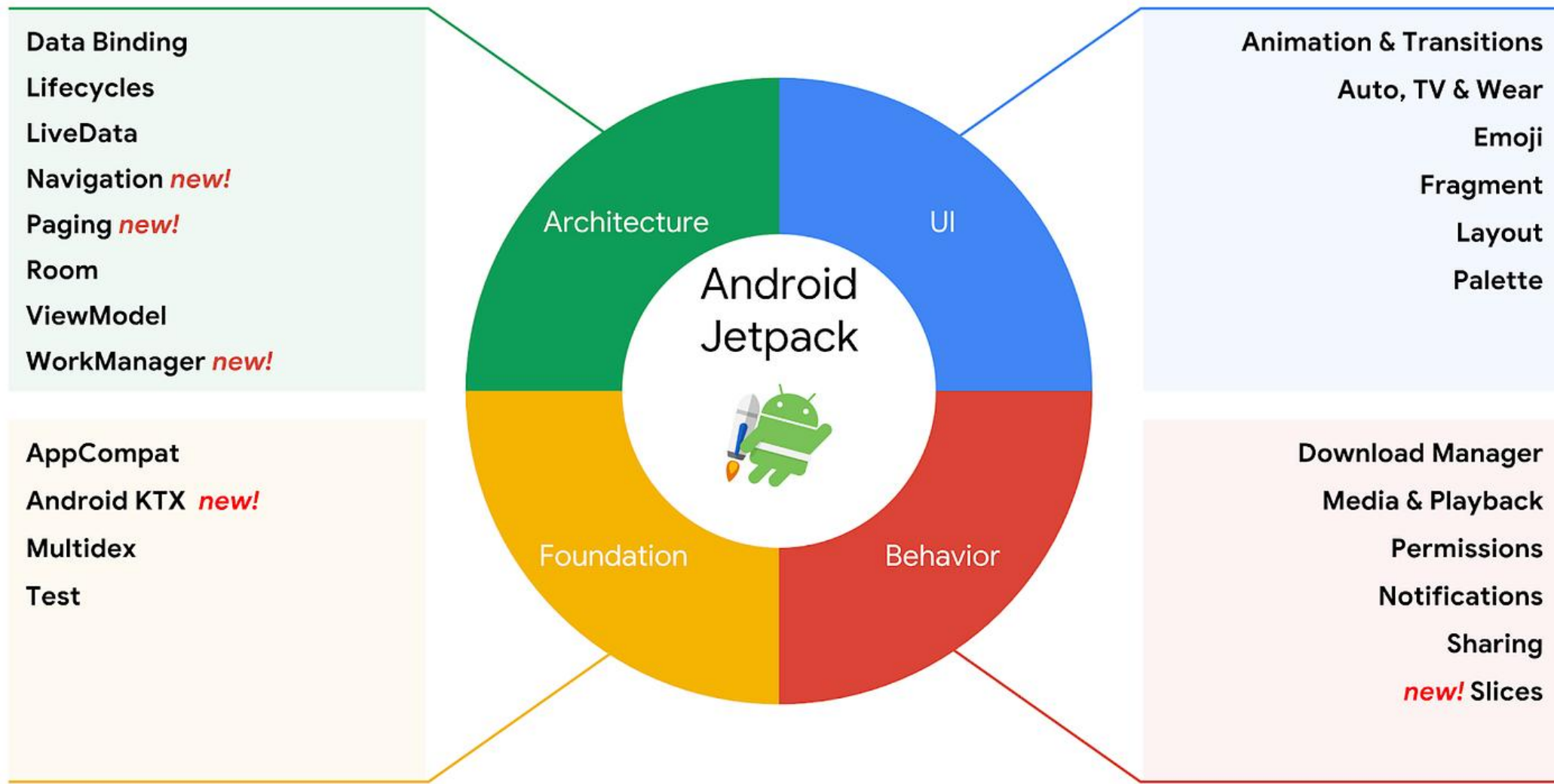
4. WorkManager

5. Demo

1. What is Architecture Components?

[Android Jetpack](#) is a key pillar of [Modern Android Development](#). It is a suite of over 100 libraries, tools and guidance to help developers follow best practices, reduce boilerplate code, and write code that works consistently across Android versions and devices so that you can focus on building unique features for your app..

2. Introduction to components in Android Jetpack



2. Introduction to components in Android Jetpack

2.1 Foundation Components

The foundation component facilitates backward compatibility, testing support, and Kotlin language support.

The foundation component consists of,

- ❖ **App compact:** Allows access to new APIs on older API versions of the platform with material design user interface implementation support.
- ❖ **Android KTX:** It is a set of Kotlin extensions included with Android Jetpack and other Android libraries.
- ❖ **Multidex:** Perform the system split of the .dex file of the application into multiple .dex files and provides support for multiple .dex files.
- ❖ **Test:** It includes the Espresso UI testing framework for runtime UI and AndroidJUnitRunner for unit testing in Android.

2. Introduction to components in Android Jetpack

2.2 Architecture Components

Architectural components facilitate highly testable robust app architecture while individually addressing developers' pain points.

Architecture components are as follows:

- ❖ **Data Binding:** A declarative format is used to connect UI elements in your layout to the data source in your application.
- ❖ **Lifecycles:** Lifecycle-aware components act in response to the change in lifecycle status of another component such as activity and fragment.
- ❖ **LiveData:** It is a lifecycle-aware data holder class that updates app component observers that are in an active lifecycle state.
- ❖ **Navigation:** Handle all aspects of in-app navigation.
- ❖ **Paging:** Gradually loads and displays small chunks of data on demand from your data source.
- ❖ **Room:** Provides Fluent database access while harnessing the power of SQLite.
- ❖ **ViewModel:** It stores and Manages UI-related data in a lifecycle conscious way and allows data to survive configuration changes such as screen rotation.
- ❖ **WorkManager:** API that makes it easy to schedule reliable, asynchronous tasks that are expected to run even if the app exits or the device restarts.

2. Introduction to components in Android Jetpack

2.3 Behavior Components

Behavioral components enable your app to integrate with modern Android services such as notification, permissions, media sharing, and assistance.

Following are the behavioral components:

- ❖ **Download Manager:** Schedule and manage long-running HTTP downloads.
- ❖ **Media & playback:** APIs for media playback and routing that are backward compatible.
- ❖ **Notifications:** Provides a backward-compatible notification API with Wear and Auto support.
- ❖ **Permissions:** Compatibility APIs for checking and requesting app permissions.
- ❖ **Preferences:** Allows to build interactive settings screens for users to configure settings.
- ❖ **Sharing:** Provides a share action suitable for an app's action bar for sharing simple data (image, text, or file).
- ❖ **Slices:** Create flexible UI elements that can display app data outside the app.

2. Introduction to components in Android Jetpack

2.4 UI Components

The UI component provides widgets and helps you to make your application attractive and adorable.

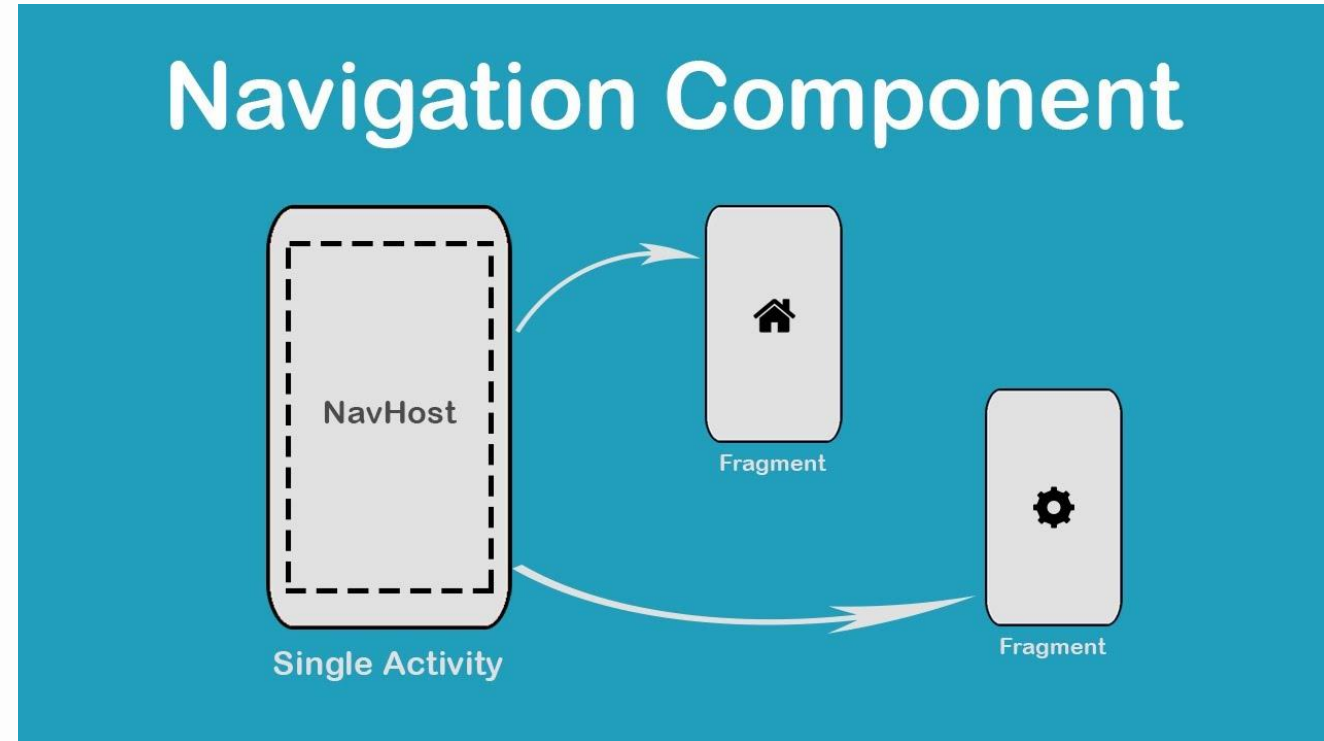
Following are the UI components:

- ❖ **Animation and transitions:** This allows you to animate UI transition, widgets on user's actions.
- ❖ **Auto:** Helps you develop an Android Auto app with Android Automotive OS.
- ❖ **Emoji:** Helps to keep Android devices up to date with the latest emoji.
- ❖ **Fragment:** A reusable unit of composable UI.
- ❖ **Layout:** Helps to build your app layout structure using Layout widgets with different algorithms.
- ❖ **Palette:** Select and apply useful information from color palettes to make your app visually appealing.
- ❖ **TV:** API helps you to build Android TV apps.
- ❖ **Wear:** Components to develop Wear apps that work on smarter screens such as smartwatches.

3. Navigation Component

3.1 Introduction to Navigation Component

The Navigation Component is a part of the Jetpack Architecture Component, designed to simplify the complexities and enhance the process of managing navigation within Android applications. Its primary purpose is to define how the app navigates between screens (fragments or activities) easily and efficiently.



3. Navigation Component

3.2 Benefits of Navigation Component

- ❖ **Simplifying Navigation:** Instead of writing code to perform transitions between screens through `FragmentManager` or `Intents`, the Navigation Component offers a declarative approach based on XML to define destinations and navigation actions. This reduces the chances of errors during navigation.
- ❖ **Integration with App Modularization:** When an app is divided into independent modules, the Navigation Component automatically creates and manages navigation graphs for each module. This allows separation of development and navigation logic for different parts of the app.
- ❖ **Automatic Graph Creation:** The Navigation Component uses information from XML files to automatically create a navigation graph. This reduces development time and establishes a clear navigation structure.
- ❖ **Effortless Back Stack Management:** The Navigation Component provides built-in back stack handling, aiding in efficiently adding, removing, and navigating between screens in the back stack.

3. Navigation Component

3.3 Structure of Navigation Component

- ❖ **Navigation Graph:** This is the core component representing the entire navigation graph of the app. The graph consists of destinations and actions between destinations.
- ❖ **NavHostFragment:** A Fragment containing a NavController object, serving as a container for fragment screens and enabling navigation based on actions and destinations from the Navigation Graph.
- ❖ **NavController:** This object manages navigation between screens based on actions and destinations defined in the Navigation Graph.

3. Navigation Component

3.4 Navigation Process in Navigation Component Navigation Graph

- ❖ **Destination:** Represents a specific screen or activity in the app. Destinations are defined in the Navigation Graph and contain information about the corresponding layout or fragment.
- ❖ **Action:** Represents a navigation interaction from one destination to another. Actions are defined in the Navigation Graph and determine how the app transitions between screens.

3. Navigation Component

3.5 Principles of Navigation

- **Starting point** – visible when application is started from a launcher, it's the last screen after returning by pressing back button
- **Navigation state is represented by a stack** (start destination at the bottom, current destination at the top of the stack)
- **Up button** takes user to hierarchical parent destination, never exits the app
- **Up button** function identically to the system **Back button** in your own app's task (when back button would not exit your app)
- **Deep link** to a destination **has got the same stack** as it would be by navigating from start destination (user is able to use Back or Up buttons to navigate through destinations back to the start destination). Any existing navigation stack is removed and replaced with the deep link's navigation stack

3. Navigation Component

3.6 Setting up navigation in a project

To include Navigation support in your project, add the following dependencies to your app's build.gradle file:

```
dependencies {
    val nav_version = "2.7.1"

    // Java language implementation
    implementation("androidx.navigation:navigation-fragment:$nav_version")
    implementation("androidx.navigation:navigation-ui:$nav_version")

    // Kotlin
    implementation("androidx.navigation:navigation-fragment-ktx:$nav_version")
    implementation("androidx.navigation:navigation-ui-ktx:$nav_version")

    // Feature module Support
    implementation("androidx.navigation:navigation-dynamic-features-fragment:$nav_version")

    // Testing Navigation
    androidTestImplementation("androidx.navigation:navigation-testing:$nav_version")

    // Jetpack Compose Integration
    implementation("androidx.navigation:navigation-compose:$nav_version")
}
```

3. Navigation Component

3.6 Setting up navigation in a project

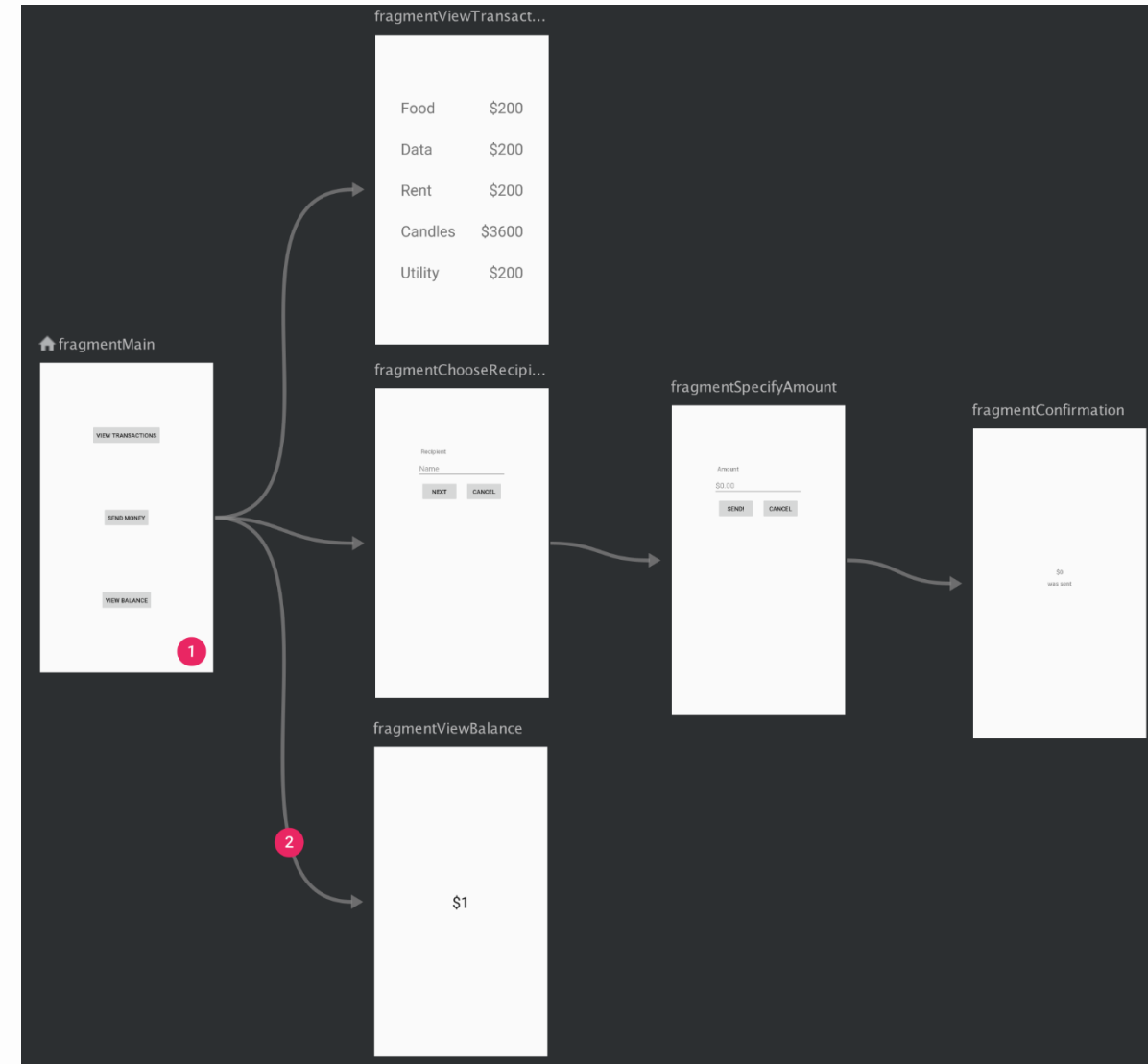
3.6.1 Create a navigation graph

Navigation occurs between your app's *destinations*—that is, anywhere in your app to which users can navigate. These destinations are connected via *actions*.

A *navigation graph* is a resource file that contains all of your destinations and actions. The graph represents all of your app's navigation paths.

(1) Destinations are the different content areas in your app.

(2) Actions are logical connections between your destinations that represent paths that users can take



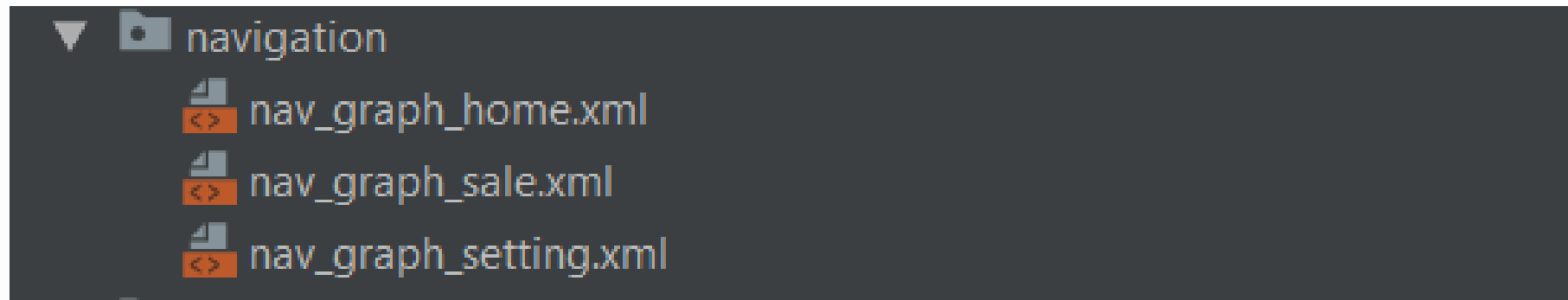
3. Navigation Component

3.6 Setting up navigation in a project

3.6.1 Create a navigation graph

Create a new resource file of type Navigation in res folder:

- navigation resource directory is created
- nav_graph.xml is created which contains navigation graph



3. Navigation Component

3.6 Setting up navigation in a project

3.6.1 Create a navigation graph

Destinations:

- Activities
- Fragments Can be created in Editor as blank destination or from existing Fragments and Activities in our project

```
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
            xmlns:app="http://schemas.android.com/apk/res-auto"
            android:id="@+id/nav_graph_sale"
            app:startDestination="@id/amountEntryFragment">

    <!-- Screen Amount Entry input -->
    <fragment android:id="@+id/amountEntryFragment"
            android:name="com.ingenico.ui.presentation.fragment.AmountEntryFragment"
            android:label="AmountEntryFragment">
    </fragment>
</navigation>
```

3. Navigation Component

3.6 Setting up navigation in a project

3.6.1 Create a navigation graph

Actions

- Actions **are scoped to the destination** they are attached to the same action on a different destination can have different behavior.
- It means **the same action can have different destination it points to** and also different attributes like transition animations

```
<fragment
    android:id="@+id/notesFragment"
    android:name="com.jshvarts.notesnavigation.presentation.notelist.NoteListFragment"
    android:label="@string/label_note_list" tools:layout="@layout/note_list_fragment">

    <action
        android:id="@+id/action_notes_to_addNote"
        app:destination="@id/addNoteFragment" />

    <action
        android:id="@+id/action_notes_to_noteDetail"
        app:destination="@id/noteDetailFragment"
        app:enterAnim="@anim/enter_slide_down"
        app:exitAnim="@anim/exit_slide_down"
        app:popEnterAnim="@anim/pop_enter_slide_up"
        app:popExitAnim="@anim/pop_exit_slide_up">
    </action>

</fragment>
```

3. Navigation Component

3.6 Setting up navigation in a project

3.6.1 Create a navigation graph

Hosting navigation: An activity hosts navigation for an application through the implementation of **NavHost** interface - Navigation Architecture Component's default is **NavHostFragment**. It is added to the activity's layout.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:defaultNavHost="true"
        app:navGraph="@navigation/mobile_navigation" />

</LinearLayout>
```

3. Navigation Component

3.6 Setting up navigation in a project

3.6.1 Create a navigation graph

NavHostFragment swaps in and out different fragment destinations as user navigates through the navigation graph.

Navigation graph is associated by **navGraph** attribute:

```
app:navGraph="@navigation/mobile_navigation"
```

To ensure that NavHostFragment intercepts the system Back button:

```
app:defaultNavHost="true"
```

To support Up button there is a need to overwrite

```
override fun onSupportNavigateUp(): Boolean {  
    return navigateUp(findNavController(this, R.id.nav_host_fragment), drawerLayout)  
}
```

3. Navigation Component

3.6 Setting up navigation in a project

3.6.1 Create a navigation graph

NavHostFragment swaps in and out different fragment destinations as user navigates through the navigation graph.

Navigation graph is associated by **navGraph** attribute:

```
app:navGraph="@navigation/mobile_navigation"
```

To ensure that NavHostFragment intercepts the system Back button:

```
app:defaultNavHost="true"
```

To support Up button there is a need to overwrite

```
override fun onSupportNavigateUp(): Boolean {  
    return navigateUp(findNavController(this, R.id.nav_host_fragment), drawerLayout)  
}
```

3. Navigation Component

3.6.1 Create a navigation graph

Navigation Controller

Navigation to a destination is done by NavController class.

NavController can be retrieved using one of the following static methods:

- NavController.findNavController(Fragment)
- NavController.findNavController(Activity, @IdRes int viewId)
- NavController.findNavController(View)
- View.findNavController()

navigate() method is used to navigate to a destination. It accepts ID of a destination or of an action.

Useful methods:

- NavController.navigateUp()
- NavController.popBackStack()

They will be translated into appropriate framework operations.

For example when we navigate() to an activity destination, the NavController calls startActivity() on our behalf.

Pressing Up and Back button calls these methods respectively to pop the top destination off the stack.

3. Navigation Component

3.6.1 Create a navigation graph

Passing data between destinations

Every destination can **define what arguments it can receive**.

Two possible ways of passing data between destinations:

- using Bundle
- type-safe way using **safeargs** Gradle plugin

To enable safeargs, add below line to build.gradle

```
apply plugin: 'androidx.navigation.safeargs.kotlin'
```

Create Bundle and pass it to the destination using navigate() method. Enable safeArgs for easy passing Data between fragment

```
<fragment
    android:id="@+id/welcomeFragment"
    android:name="com.ddona.jetpack.ui.nav.WelcomeFragment"
    android:label="{username}">
    <argument
        android:name="username"
        app:argType="string" />
    <argument
        android:name="password"
        app:argType="string" />
    <action
        android:id="@+id/action_welcomeFragment_to_homeFragment"
        app:destination="@id/homeFragment" />
</fragment>
```

```
val username = binding.edtUsername.text.toString()
val password = binding.edtPassword.text.toString()
val action = LoginFragmentDirections
    |.actionLoginFragmentToWelcomeFragment(username, password)
findNavController().navigate(action)
```

```
private val args: WelcomeFragmentArgs by navArgs()

override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View {
    binding = FragmentWelcomeBinding
        |.inflate(inflater, container, attachToParent: false)
    binding.tvUserName.text = args.username
    binding.tvPassword.text = args.password
```

3. Navigation Component

3.6.1 Create a navigation graph

Nested navigation graph

- A series of destinations can be grouped into a **sub-graph** within a navig
- Useful for **organizing and reusing** sections of app's UI like a login flow.
- Nested graph must have a **start destination**.
- Root graph only accesses the nested graph **through its starting destin**
- In navigation graph you can reference other graph by using <include>

```
<include app:graph="@navigation/nav_graph_sale" />
```


3. Navigation Component

3.6.1 Create a navigation graph

Deep linking to a destination

- Deep links allow to **jump in the middle** of app's navigation. It can be from an **url link** or **pending intent**.
- Navigation library ensures users start on the appropriate destination with the **appropriate back stack**.
- When user uses the Back button they navigate back up the navigation stack just as though they entered app from app's entry point.
- When deep link is triggered, the task back stack is cleared and replaced with the deep link destination

```
<fragment
    android:id="@+id/loginFragment"
    android:name="com.ddong.jetpack.ui.nav.LoginFragment"
    android:label="LoginFragment">
    <action
        android:id="@+id/action_loginFragment_to_welcomeFragment"
        app:destination="@id/welcomeFragment" />
    <argument
        android:name="username"
        android:defaultValue="@null"
        app:argType="string"
        app:nullable="true" />
    <!--This will open when any deep link request:
    for example: open google search on launcher and
    enter the deep link: doanpt.com/login/DoanPT1
    then your app will display on App list result-->
    <deepLink
        android:id="@+id/deepLink2"
        app:uri="doanpt.com/login/{username}" />
</fragment>
```

```
<activity android:name=".ui.nav.NavScreenActivity">
    <nav-graph android:value="@navigation/nav_graph" />
</activity>
```

4. WorkManager

4.1 Introduction to WorkManager

WorkManager is a component within the Jetpack Architecture Component suite that facilitates managing and executing asynchronous tasks in Android applications. It addresses the challenge of ensuring tasks are executed reliably, even in scenarios where the app might be in the background, terminated, or the device is restarted.

4. WorkManager

4.2 Why WorkManager is Needed

- ❖ **Ensuring Continuity:** WorkManager guarantees that tasks are executed even if the app is in the background, terminated, or the device restarts. This ensures that important tasks, such as data synchronization or notifications, are not missed.
- ❖ **Managing Execution Time:** WorkManager allows you to set constraints on when tasks should be executed, such as requiring a network connection or device charging. This ensures that tasks are executed when the conditions are optimal.
- ❖ **Easy Integration:** WorkManager is designed to seamlessly integrate with Android versions from 4.0 (API level 14) and higher, providing a consistent approach to background processing across different devices.

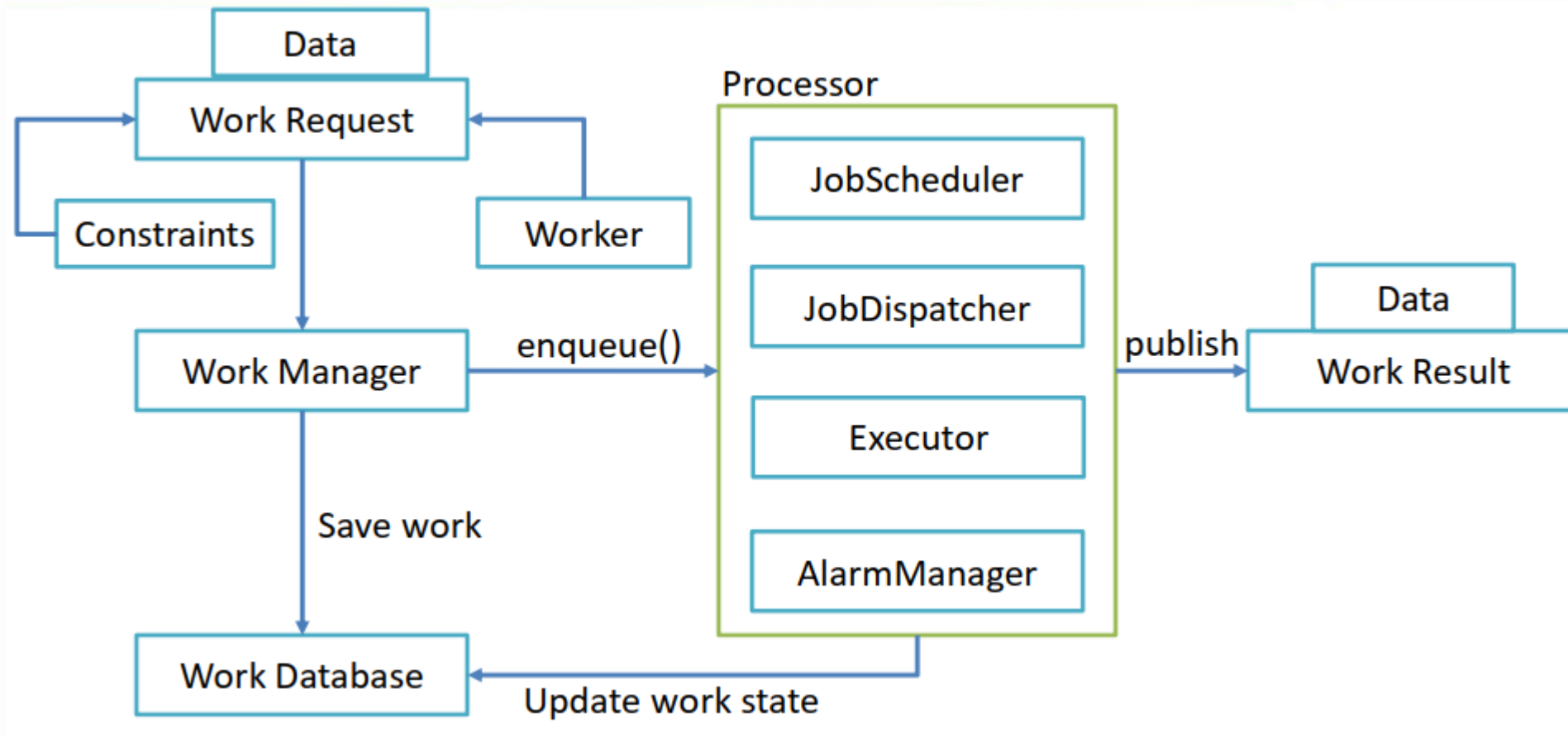
4. WorkManager

4.2 Structure of WorkManager

- ❖ **Work:** This represents a single unit of work that needs to be executed asynchronously. It could be a network call, database operation, or any other task that runs in the background.
- ❖ **Worker:** A Worker is an implementation of the actual work that needs to be done. Developers create a subclass of the Worker class and override its `doWork()` method to define the work to be performed.
- ❖ **Constraints:** Constraints are conditions that must be met before a work request is executed. They can include factors like network connectivity, charging status, or the device being idle.
- ❖ **WorkRequest:** A WorkRequest defines a set of input data, constraints, and the Worker class to use. It's used to enqueue a piece of work to be executed.

4. WorkManager

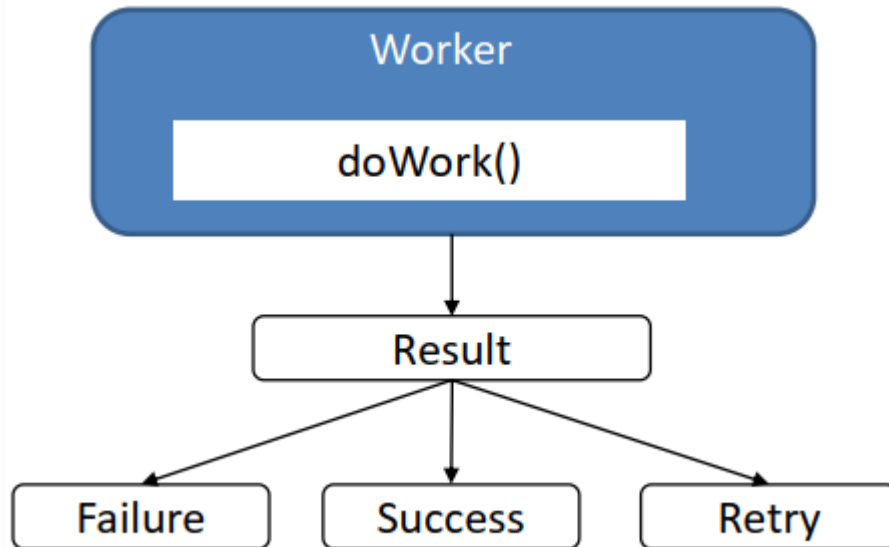
4.2 Structure of WorkManager



4. WorkManager

4.2 Structure of WorkManager – Worker

This is where you put the code for the actual work you want to perform in the background. You'll extend this class and override the `doWork()` method



```
private const val TAG = "BlurWorker"
class BlurWorker(ctx: Context, params: WorkerParameters) : Worker(ctx, params) {

    override fun doWork(): Result {
        val appContext = applicationContext

        makeStatusNotification("Blurring image", appContext)

        return try {
            val picture = BitmapFactory.decodeResource(
                appContext.resources,
                R.drawable.android_cupcake)

            val output = blurBitmap(picture, appContext)

            // Write bitmap to a temp file
            val outputUri = writeBitmapToFile(appContext, output)

            makeStatusNotification("Output is $outputUri", appContext)

            Result.success()
        } catch (throwable: Throwable) {
            Log.e(TAG, "Error applying blur")
            Result.failure()
        }
    }
}
```

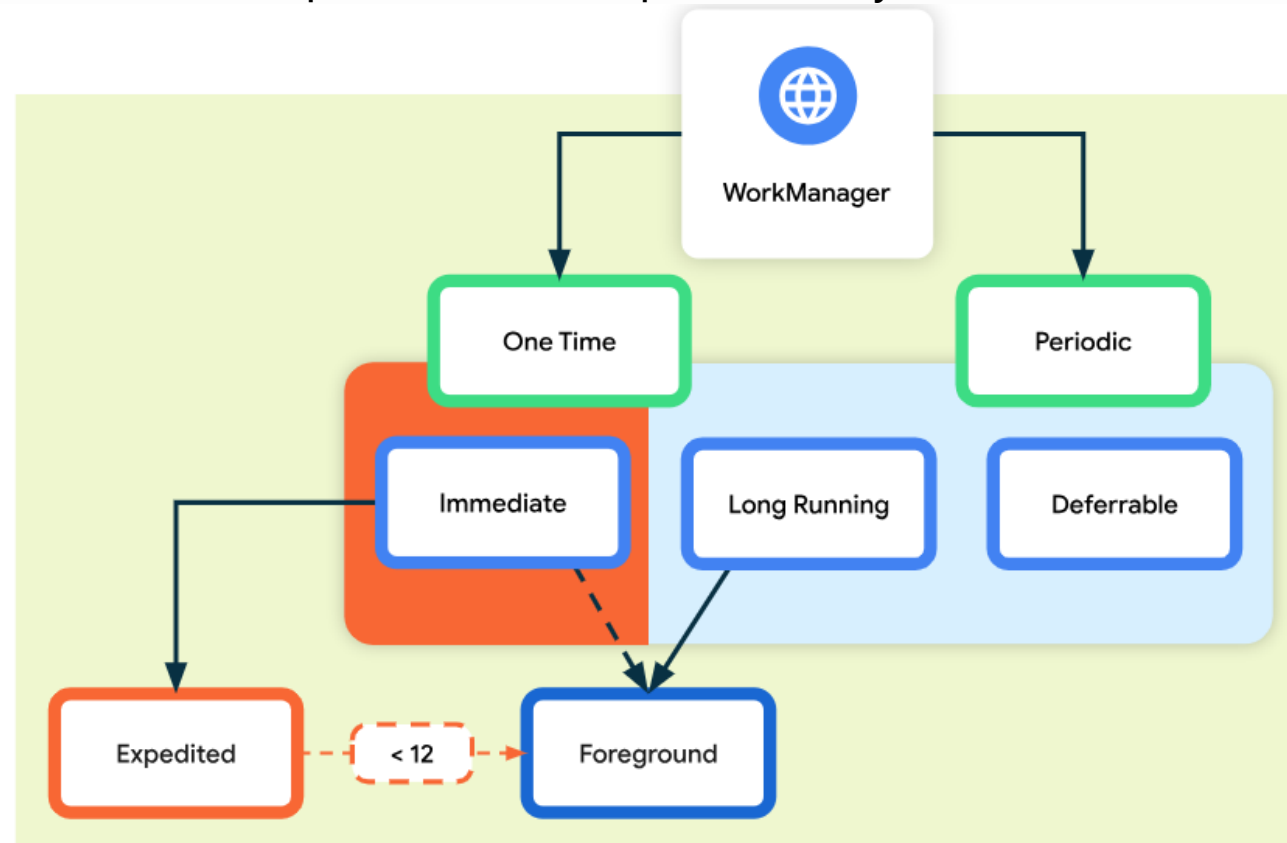
4. WorkManager

4.2 Structure of WorkManager – Work Request

WorkRequest is class to define how and when we should start work and it's should be enqueued in WorkManager There are 2 type of work request:

OneTimeWorkRequest: A WorkRequest that will only execute once

PeriodicWorkRequest: A Work request that will repeat on a cycle



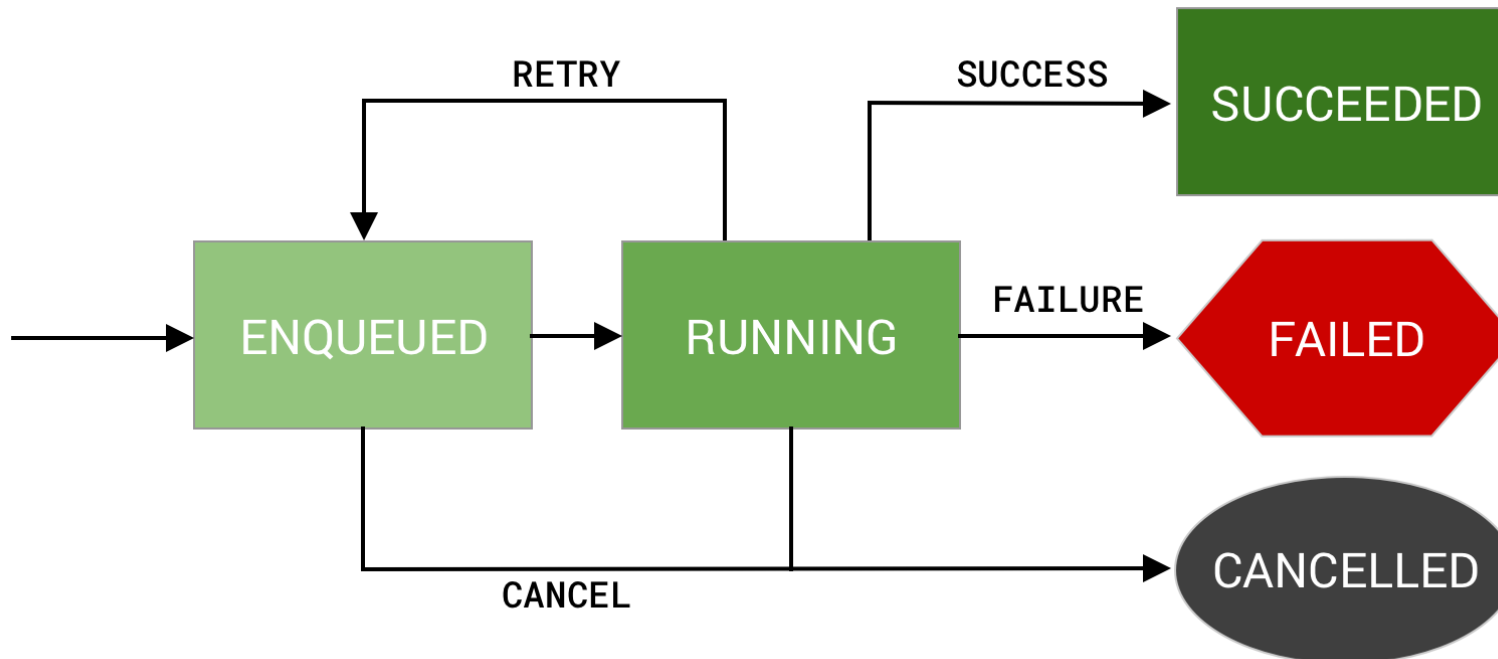
4. WorkManager

4.2 Structure of WorkManager – Work Request

One-time work states

For a **one-time** work request, your work begins in an **ENQUEUED** state.

In the **ENQUEUED** state, your work is eligible to run as soon as its **Constraints** and initial delay timing requirements are met. From there it moves to a **RUNNING** state and then depending on the outcome of the work it may move to **SUCCEEDED**, **FAILED**, or possibly back to **ENQUEUED** if the result is **retry**. At any point in the process, work can be cancelled, at which point it will move to the **CANCELLED** state.



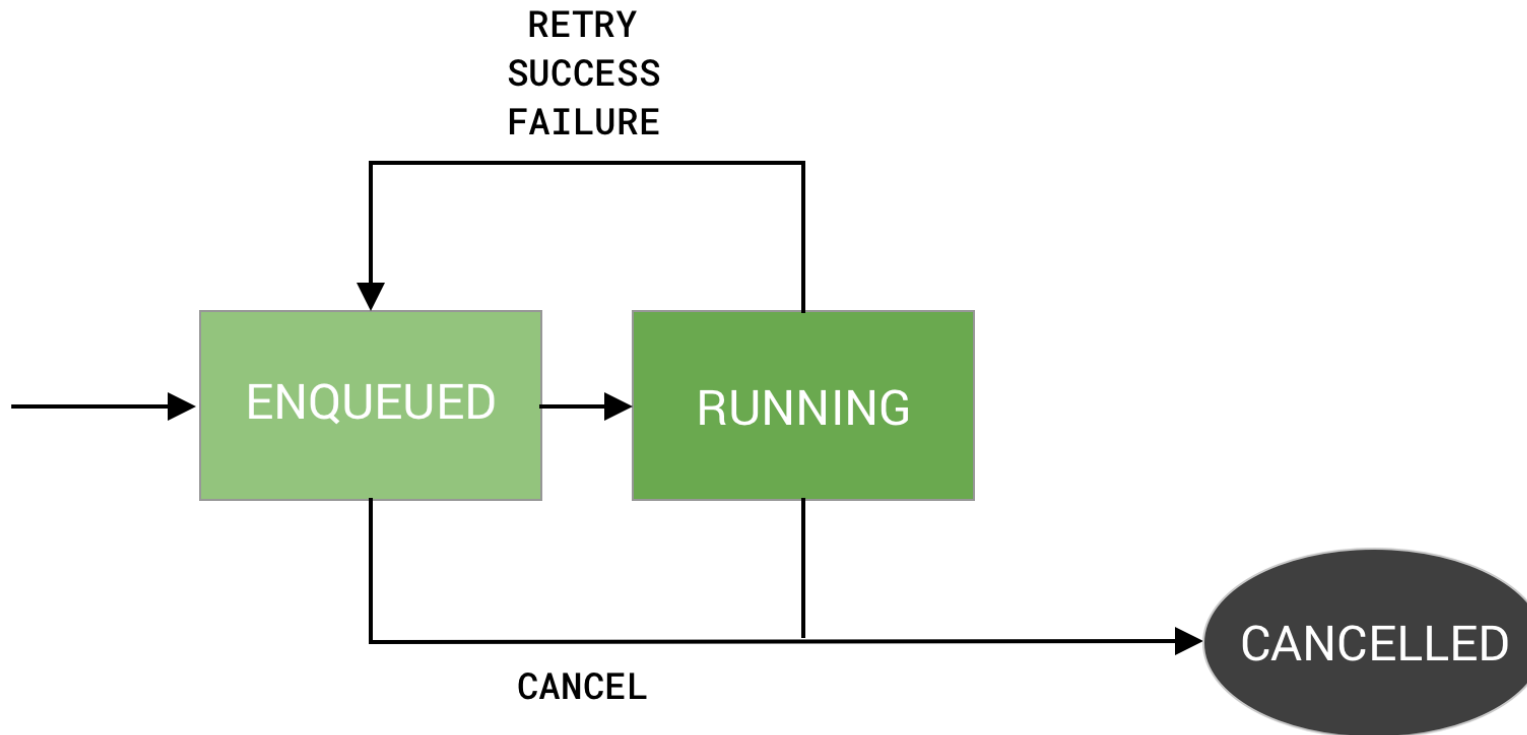
State diagram for one-time work. **SUCCEEDED**, **FAILED** and **CANCELLED** all represent a terminal state for this work. If your work is in any of these states, `WorkInfo.State.isFinished()` returns true.

4. WorkManager

4.2 Structure of WorkManager – Work Request

Periodic work states

Success and failed states apply only to one-time and chained work. For periodic work, there is only one terminal state, CANCELLED. This is because periodic work never ends. After each run, it's rescheduled, regardless of the result. Figure 2 depicts the condensed state diagram for periodic work.



4. WorkManager

4.2 Structure of WorkManager – Work Request

Blocked state

There is one final state we haven't mentioned yet, and that is BLOCKED. This state applies to work that is orchestrated in a series, or chain of work. Work chains, and their state diagram, are covered in Chaining work.

4. WorkManager

4.2 Structure of WorkManager – Work Constraints

- WorkManager constraints specify the requirement that needs to be met before being executed task. In other words, you can say constraints specify the condition for running task under that specific condition. These constraints can be related to storage, battery or network
- There are some type of Constraints provides by Workmanager:
 - `requiresStorageNotLow()`
 - `requiresBatteryNotLow()`
 - `requiresCharging()`
 - `requiresDeviceIdle()`
 - `getRequiredNetworkType()`: you can specify the NetworkType

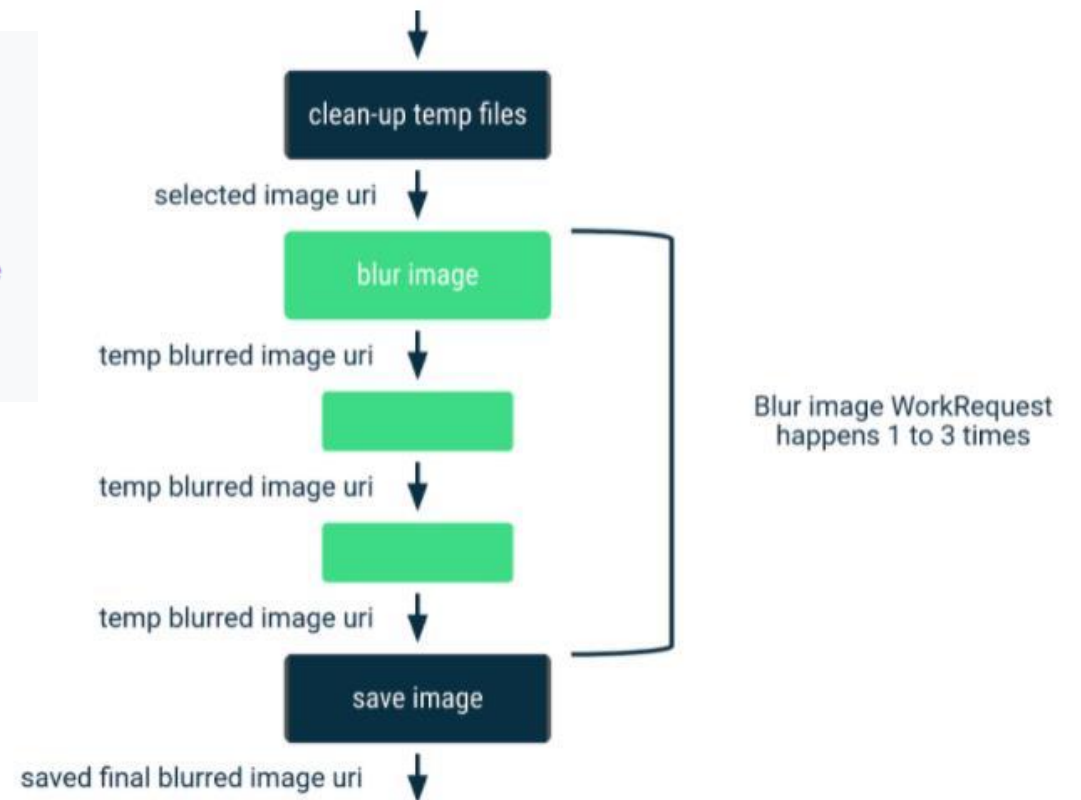
4. WorkManager

4.2 Structure of WorkManager – Chain work

- WorkManager allows you to create separate WorkerRequests that run in order or parallel. In this step you'll create a chain of work that looks like this:

```
WorkManager.getInstance(myContext)
    .beginWith(workA)
    // Note: WorkManager.beginWith() returns a
    // WorkContinuation object; the following calls are
    // to WorkContinuation methods
    .then(workB)    // FYI, then() returns a new WorkContinuation instance
    .then(workC)
    .enqueue()
```

```
WorkManager.getInstance(myContext)
    // First, run all the A tasks (in parallel):
    .beginWith(Arrays.asList(workA1, workA2, workA3))
    // ...when all A tasks are finished, run the single B task:
    .then(workB)
    // ...then run the C tasks (in parallel):
    .then(Arrays.asList(workC1, workC2))
    .enqueue()
```



4. WorkManager

4.2 Structure of WorkManager – Work continuation

- You can create more complex sequences by joining multiple chains with the `WorkContinuation.combine(List<OneTimeWorkRequest>)` methods. For example, suppose you want to run a sequence like this

```
val chain1 = WorkManager.getInstance(myContext)
    .beginWith(workA)
    .then(workB)
val chain2 = WorkManager.getInstance(myContext)
    .beginWith(workC)
    .then(workD)
val chain3 = WorkContinuation
    .combine(Arrays.asList(chain1, chain2))
    .then(workE)
chain3.enqueue()
```

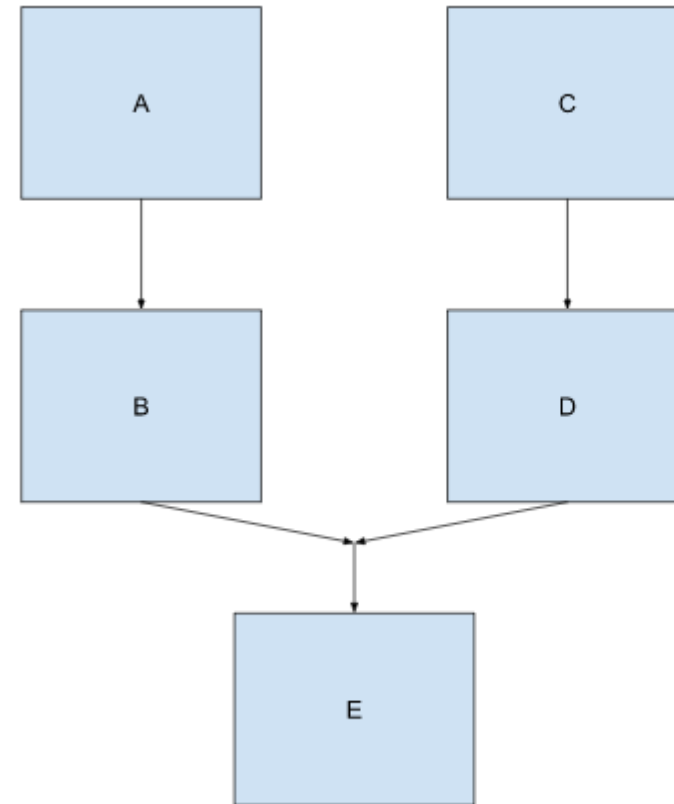


Figure 1. You can use `WorkContinuation` to set up complex chained tasks.

4. WorkManager

4.2 Structure of WorkManager – Cancelling work

- If you no longer need your previously enqueued work to run, you can ask for it to be cancelled. Work can be cancelled by its name, id or by a tag associated with it

```
// by id
workManager.cancelWorkById(syncWorker.id)

// by name
workManager.cancelUniqueWork("sync")

// by tag
workManager.cancelAllWorkByTag("syncTag")
```

4. WorkManager

4.2 Structure of WorkManager – Cancelling work

- If you no longer need your previously enqueued work to run, you can ask for it to be cancelled. Work can be cancelled by its name, id or by a tag associated with it

```
// by id
workManager.cancelWorkById(syncWorker.id)

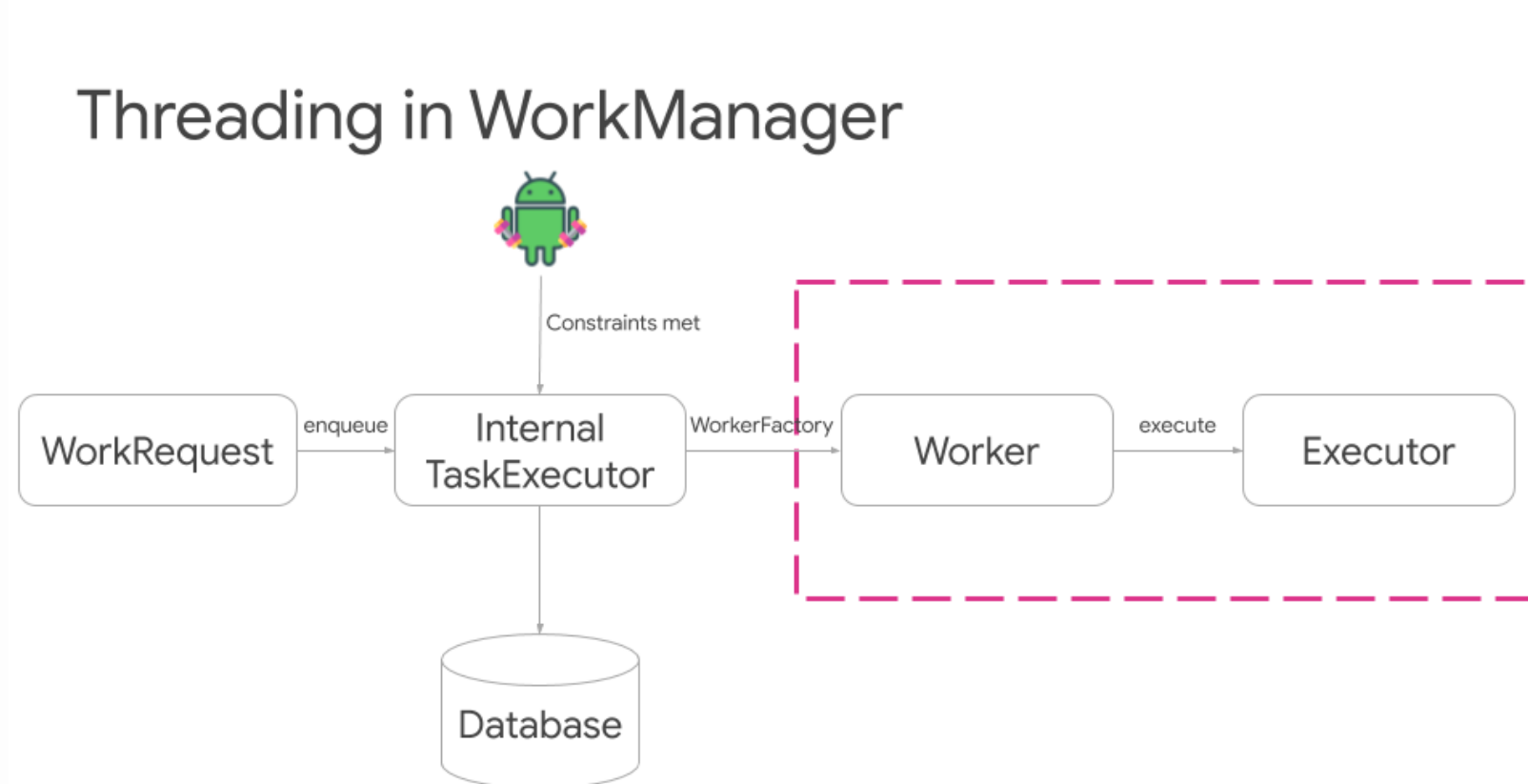
// by name
workManager.cancelUniqueWork("sync")

// by tag
workManager.cancelAllWorkByTag("syncTag")
```

4. WorkManager

4.2 Structure of WorkManager – Threading in WorkManager

- When you use a Worker, WorkManager automatically calls Worker.doWork() on a background thread. The background thread comes from the Executor specified in WorkManager's Configuration. By default, WorkManager sets up an Executor for you - but you can also customize your own.





Thank You !

OPEN MIND - OPEN DOORS